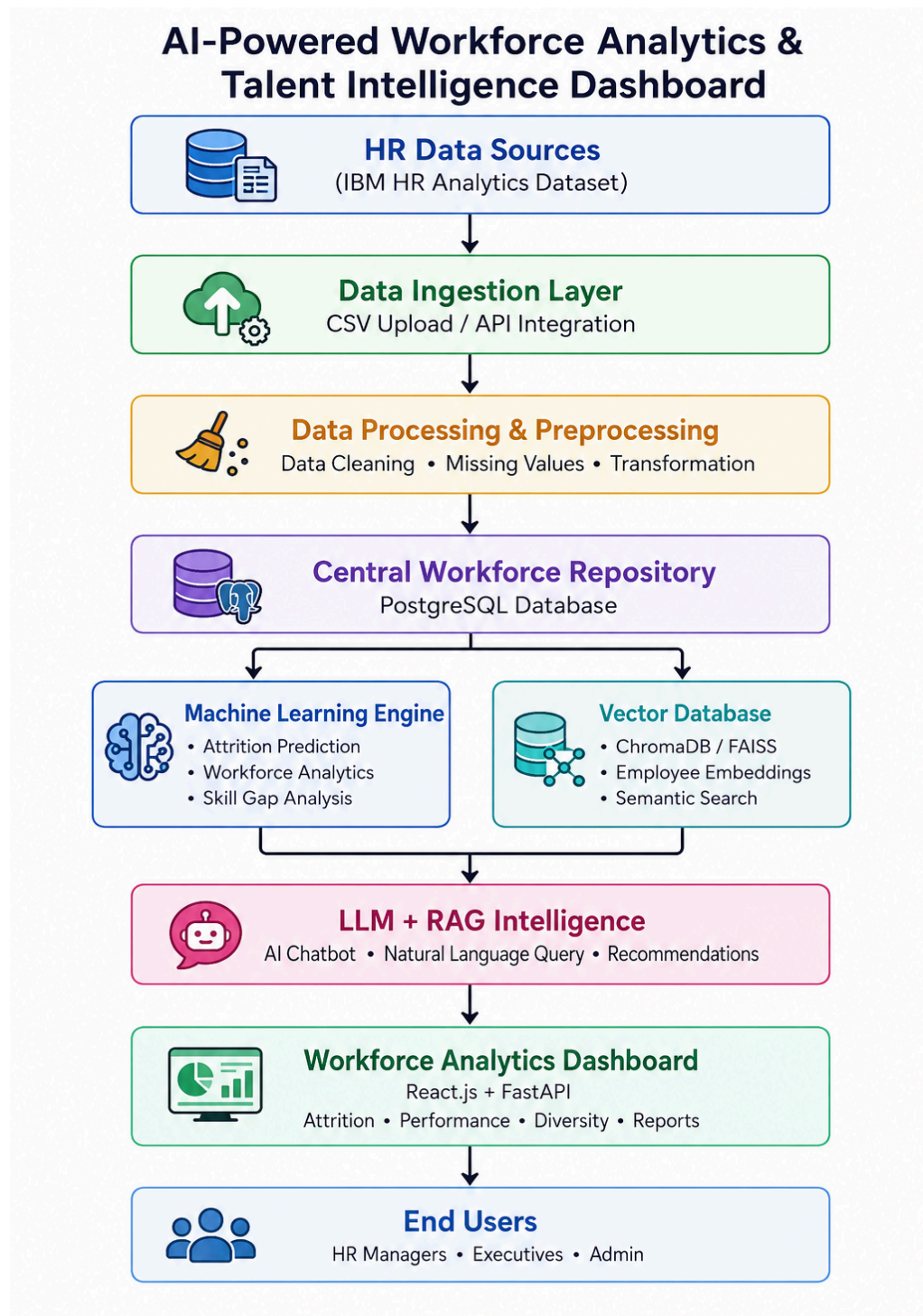


Milestone 1: Repository Architecture

Prepared by: Gowshika T

1. System Architecture Diagram



2. One-Page Explanation: Repository Architecture

The repository architecture of the AI-Powered Workforce Analytics & Talent Intelligence Dashboard follows a layered design to efficiently process, analyze, and visualize workforce data.

The system begins by collecting employee information from the IBM HR Analytics Employee Attrition Dataset, which acts as the primary data source for this implementation. The collected data is passed through the Data Ingestion Layer, where it is imported into the system through CSV uploads or APIs.

The data then moves to the **Data Processing Layer**, where preprocessing techniques such as data cleaning, missing value handling, duplicate removal, and data transformation are performed to improve data quality. After preprocessing, the cleaned data is stored in a Central Workforce Repository using PostgreSQL, which acts as the primary database for employee, performance, training, and attrition information.

The stored data is then utilized by Machine Learning models to predict employee attrition, analyze workforce performance, identify skill gaps, and generate workforce health insights. To enable intelligent search and conversational analytics, employee documents and organizational information are converted into vector embeddings and stored in a Vector Database (ChromaDB).

These embeddings are used by the LLM + Retrieval-Augmented Generation (RAG) module to answer HR-related questions and generate AI-powered recommendations. Finally, all analytical insights, predictions, reports, and chatbot responses are presented through a React.js-based Workforce Analytics Dashboard. The dashboard provides HR managers, administrators, and business leaders with real-time workforce intelligence, enabling data-driven decision-making for employee retention, workforce planning, and organizational growth.

3. Technology Stack

Component	Technology
Dataset	IBM HR Analytics Employee Attrition
Frontend	React.js
Backend	FastAPI
Database	PostgreSQL
Machine Learning	Scikit-learn, XGBoost
Vector Database	ChromaDB / FAISS
LLM	Llama 3 / OpenAI GPT
RAG Framework	LangChain
Visualization	Chart.js / Recharts
Deployment	Docker

4. Suggested Repository Structure

The following folder hierarchy is proposed to maintain a modular and scalable codebase:workforce-analytics-dashboard/

|

|— dataset/ # Raw and processed IBM HR data

|— frontend/ # React.js source code and assets

|— backend/ # FastAPI server and business logic

|— database/ # SQL schemas and migration scripts

|— ml_models/ # Predictive models and training scripts

|— vector_db/ # ChromaDB/FAISS storage and indexing

|— rag/ # LangChain implementation and LLM prompts

|— api/ # Endpoint definitions and documentation

|— docs/ # Technical documentation and project reports

|— docker/ # Dockerfiles and orchestration configs